

Week 1 Assignment — Days 1–5

Practical Cyber Security (v2) · Week 1: Building the foundation

This handout is everything you have to **do and hand in** for Week 1: one homework task after each class (Days 1–4), the Day 5 build task, and the weekend assignment. Class notes, recaps and key terms are in your Student Pack.

How the week fits together

Each day answers one question. The weekend assignment puts all five answers into one story.

Day	The question	The path
1	What are we protecting, and what's the line we don't cross?	CIA → threats → authorization
2	What is actually running on the machine?	program → executable → process → CPU/RAM → OS
3	How does the machine talk to others?	IP → MAC → gateway → router (NAT) → ISP → Internet
4	How does a web request work?	DNS → TCP → TLS → HTTP → server
5	How is information protected?	encryption → hashing → passwords → certificates → signatures
Weekend	Can you follow one login end to end — and say where it can fail?	integrate → threat-model → prove it

The pattern for every day

Every homework follows the same six steps, so you always know what you are doing and why:

Learn (class) → **Do** (build or draw it) → **Investigate** (look at the real thing) → **Secure** (answer the day's security question) → **Evidence** (prove it on your machine) → **Commit** (push to your repo).

Every day also ends with one **Real job** question: where would you meet this in a real security job? Two or three sentences is plenty.

Your course repo. You create one private Git repo this week (Day 1). After every class you add three things: today's attack, today's defence, today's artifact. By Day 20 it is your portfolio.

When things are due

Task	Due
Day 1 homework	Start of Day 2
Day 2 homework	Start of Day 3
Day 3 homework	Start of Day 4
Day 4 homework	Start of Day 5
Day 5 homework + weekend assignment	Before Monday, start of Day 6 (one PDF)

Day 1 — What are we protecting? · due start of Day 2

Security question: What could go wrong if this asset is compromised?

Do — set up your course repo

1. Create a free account at github.com **or** gitlab.com (turn on 2-factor auth).
2. Create a **private** repository named `cybersec-course`.
3. Install Git: Windows → <https://git-scm.com/download/win> · Linux → `sudo apt install git` · macOS → `xcode-select --install`.
4. `git clone <your repo URL>` then `cd cybersec-course`.

5. Create `README.md` (see below), then: `git add . && git commit -m "day 1" && git push`
6. Structure to build over the course: `README.md`, `day01/`, `day02/`, ... one folder per day.

Stuck? Bring your laptop to the start of Day 2 — we'll fix it in the first 5 minutes.

Investigate — pick one asset of your own (your email account, your phone, your college login — something real). In `day01/asset.md`, fill in:

<u>Asset</u>	<u>Which CIA pillar matters most, and why?</u>	<u>If it were compromised, what could go wrong?</u>

Secure — write `README.md`. Two short paragraphs: **why** you're taking this course, and the sentence *"I will only test systems I am authorized to test."* Start a "Today I learned" list. Then read and sign the **Authorization Pledge** below and bring it to Day 2.

Evidence — the signed pledge, plus your repo showing its first commit **including** `day01/asset.md` (`git add . && git commit -m "day 1" && git push`).

Real job — Why does a penetration tester need written scope before touching anything?

AUTHORIZATION PLEDGE

I understand that accessing or testing computer systems without authorization is unlawful under the Information Technology Act, 2000, regardless of intent or outcome. I will only test systems I am authorized to test — my own isolated lab, or systems for which I hold explicit written permission or a valid bug-bounty scope. I will report vulnerabilities responsibly and will not use anything taught in this course to cause harm.

Name: _____ Roll no: _____ Signature: _____
Date: _____

Day 2 — What is running on the machine? · due start of Day 3

Security question: How could an attacker abuse a running process?

Do

1. Draw the **source code** → **executable** → **running process** pipeline yourself. Label where the **CPU**, **RAM**, and **disk** each come in.
2. One paragraph: **what is the difference between a program and a process?**

Reminder of the model from class: you write source code → a compiler turns it into machine code (an executable file on disk) → the OS loads it into RAM and points the CPU at it → it's now a running process, identified by a PID.

Investigate — Task Manager → Details (Windows) or `htop` (Linux). Browsers run as *many* processes, so pick **one** browser process (e.g. one `chrome.exe`) and note its **PID** and **memory**. Then open ten tabs and note what happens to the memory (of that process or of the browser in total — say which).

Secure — in 2–3 sentences, answer the security question. Use class ideas: a process runs with the permissions of whoever started it; instructions and data share the same RAM.

Evidence — a screenshot of the browser process with its PID and memory visible.

Commit — everything above into `day02/`, and update your “today I learned” list.

Real job — How could a SOC analyst use process information during an incident investigation?

Optional extras (not graded):

- *Process tracking in Event Viewer (Windows, admin)* → see the separate handout `week1-optional-lab-event-4688`.
- *WSL setup* → below.

OPTIONAL — EXPLORE AND SET UP WSL (WINDOWS)

A real Linux kernel running next to your Windows kernel, on the same machine — the live version of today’s “kernel” idea. It also lets you run the Linux commands used all course (`htop`, `ip a`, `dig`, ...) yourself instead of only reading the Windows equivalent.

1. Open PowerShell **as Administrator** → `wsl --install` . Restart if prompted.
2. After restart, Ubuntu launches automatically and finishes installing — create a UNIX username and password (separate from your Windows login; the typed password won’t show characters, that’s normal).
3. Check it worked: on Windows, `wsl --status` ; inside Ubuntu, `uname -a` (should mention `microsoft-standard-WSL2`).
4. Inside Ubuntu: `sudo apt update && sudo apt install -y htop` , then run `htop` . Compare it side by side with Task Manager — same idea (processes, PID, memory), a completely separate kernel managing it.
5. One sentence: is the Ubuntu you just opened a separate physical computer, a separate virtual machine, or something else?

(Not the same as the Kali attacker VM used later for Days 12–20 labs — WSL2 isn’t used there; see `LAB-SETUP.md` . This is just to get comfortable with a Linux shell early.)

Day 3 — How does the machine communicate? · due start of Day 4

Security question: Where could an attacker intercept or redirect traffic?

Do — draw your home network in this shape, using your own devices and real values:

```

Your laptop      MAC:      Private IP:
_____|
_____| local hop – the MAC address is used here
      v
Wi-Fi router     Default gateway:      NAT happens here
_____|
_____| Public IP:
      v
ISP → Internet → Web server
  
```

Add every other device on your network (phone, TV, ...) with its private IP — if you can't find one, write "unknown" (the router's device list often shows them). *On a hostel, college or mobile-hotspot network you may be behind an extra layer of NAT, so the router's public IP may not match what a "what's my IP" site shows — draw what you can and note it. Hint: `ip a` / `ipconfig /all` gives your IP, MAC and gateway; a "what's my IP" site shows the router's public IP.*

Investigate — answer these four questions in your own words:

1. Why does the laptop need a **MAC address** on the local network?
2. Why does **your laptop** need a **default gateway**? What would break without it?
3. **Where does NAT happen**, and what does it change?
4. Does the **web server** see **your laptop's private IP**? Explain.

Then two sentences: **why does** `whatsmyip.com` **show a different address than** `ipconfig` / `ip a` ?

Secure — in 2–3 sentences, answer the security question. Use class ideas: anyone on your local network can see your traffic, and ARP spoofing can put an attacker in the middle even on a switch. What is the real defence?

Evidence — screenshot your `ip a` / `ipconfig /all` output (IP, MAC, gateway) and a `ping` and `tracert` (Windows: `tracert`) to a public site. *(These also count for Part C of the weekend assignment.)*

Commit — into `day03/`, and update your "today I learned" list.

Real job — Why would a security engineer care whether an address is private or public?

Day 4 — How does a web request work? · due start of Day 5

Security question: What can an attacker learn from an HTTP request?

Do — write the 7-step story of loading `example.com`, from keypress to rendered page, in your own words. *(The 7 steps from class: type the URL → DNS → TCP → TLS → HTTP → render → repeat for every image/script. Part A of the weekend assignment extends this into a full login story, so do it properly now.)*

Investigate — dissect one real request. In DevTools → Network, log in to an account **you own** (a throwaway one is ideal — never someone else's) and pick the login request. Tips: tick **Preserve log** so the request survives the page redirect, and try the **Fetch/XHR** filter if you can't spot it. Fill in this table from what you see:

Field	What you found
<u>HTTP method</u>	
<u>URL</u>	
<u>Host</u>	
<u>User-Agent</u>	
<u>Content-Type</u>	
<u>Cookie</u>	<i>(blur the value)</i>
<u>Authorization</u> header	<i>(if present — blur the value; write “not present” otherwise)</i>
<u>Status code</u>	
<u>Response headers worth noting</u>	<i>(e.g. <u>Set-Cookie</u>, <u>Content-Type</u>)</i>
<u>HTTP or HTTPS?</u>	

Then answer: your browser can see every field above. Which of them would be hidden from someone watching your Wi-Fi (because of TLS), and what could that person still see?

Secure — answer the security question in 2–3 sentences, and one more: what does the padlock guarantee, and what does it not?

Evidence — a screenshot of the request line + headers, with credentials blurred. Never submit a screenshot showing a real password or token.

Commit — into `day04/`, and update your “today I learned” list.

Real job — Why does a penetration tester inspect HTTP headers?

Day 5 — How is information protected? · due with the weekend assignment

Security question: Which cryptographic primitive protects which security property?

BACKGROUND — WHAT CRYPTOGRAPHY IS

Cryptography is the mathematical practice of scrambling readable data into an unreadable format, so that information stays secure across networks and devices.

The four goals

<u>Goal</u>	<u>What it means</u>	<u>In class we called it</u>
<u>Confidentiality</u>	<u>Keeps data private: only authorized users with the decryption key can read it.</u>	<u>secrecy</u>
<u>Integrity</u>	<u>Ensures data has not been altered or tampered with in storage or in transit.</u>	<u>tamper-evidence</u>
<u>Authentication</u>	<u>Verifies the true identity of users, systems or devices.</u>	<u>identity</u>
<u>Non-repudiation</u>	<u>Proves a specific party sent a message or signed a transaction, so they cannot deny it later.</u>	<u>signatures</u>

The three main types

- **Symmetric cryptography** — one secret key both locks (encrypts) and unlocks (decrypts) the data. Fast and efficient for bulk data. Example: **AES**.
- **Asymmetric cryptography** — a mathematically linked *pair* of keys: a public key encrypts, the private key decrypts. Example: **RSA**. (*Signing runs the other way round: the private key signs, the public key verifies.*)
- **Hash functions** — turn data into a fixed-size, unique digital fingerprint, so any change is detectable. A hash is one-way and has no key, so it is not “scrambling” you can undo. Hashes are used for password storage, but a plain hash is not enough: sites store a **salted, slow** hash (bcrypt / Argon2), as in class.

Where you already meet it

- **Secure web browsing** — SSL/TLS secures HTTPS traffic in online banking and shopping.
- **End-to-end encryption** — messaging tools like WhatsApp protect private conversations from being intercepted.
- **Password hashing** — websites store password hashes instead of plain text, to protect user accounts.

Do — build the 5-primitives table in `day05/primitives.md`. Use your own words; a definition copied from a website earns nothing. In the *Security problem it solves* column use the four goals from the background above (confidentiality, integrity, authentication, non-repudiation) — or name the specific attack it stops.

<u>Primitive</u>	<u>What it does</u>	<u>Security problem it solves</u>	<u>One real use</u>
<u>Symmetric encryption</u>			
<u>Asymmetric encryption</u>			
<u>Hash</u>			
<u>Salted slow hash</u> (<u>passwords</u>)			
<u>Signature / certificate</u>			

Investigate — two things from class, on your own machine:

1. Browser padlock → certificate details: who issued it, when does it expire, what is the chain up to a root CA?
2. Hash a short text file (`sha256sum` / `Get-FileHash`), change **one letter**, hash again.

Secure — in 2–3 sentences: **why must a website never store your password as plaintext or “encrypted”, and what should it store instead?**

Evidence — a screenshot of the certificate chain, and of the two different hashes.

Commit — into `day05/`, and update your “today I learned” list.

Real job — Why does a security engineer need to tell hashing and encryption apart?

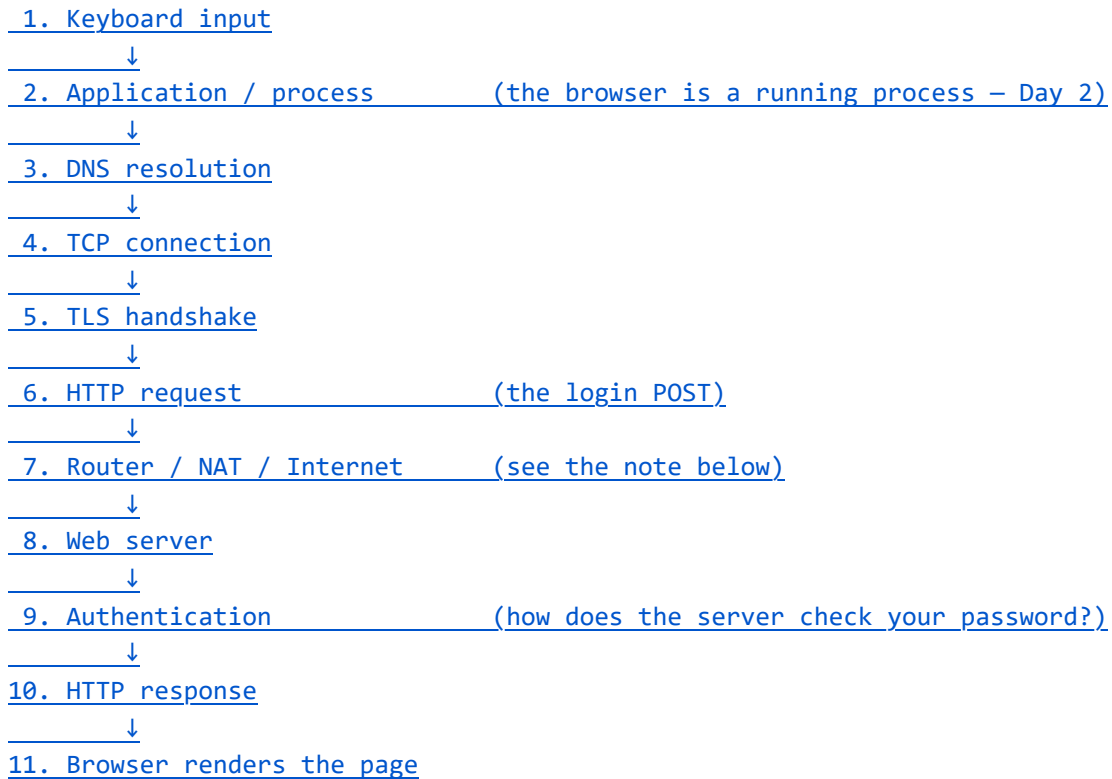
Weekend assignment — Week 1

Submit **one PDF** before Monday (start of Day 6).

This is the integration exercise: everything from Days 1–5, in one investigation.

PART A — INTEGRATE: ONE LOGIN, END TO END

Explain **what happens when you log in to a website**, from pressing a key to the server checking your password. Your story **must cover these eleven stages, in this order**:



Note on stage 7: your packets pass through the router (and NAT) from stage 3 onward, not only at stage 7. Use stage 7 to explain what the router and NAT do to those packets on their way out and back.

For each stage, write (a) what happens and (b) at least one security consideration — one or two sentences each is enough, in your own words. Aim for about two pages, plus a diagram (the diagram can replace some of the words).

PART B — R&D STRETCH (RESEARCH SOMETHING WE DID NOT COVER)

Pick **one**:

- Explain **DNS-over-HTTPS (DoH)**: what problem it solves, and one reason it's controversial.
- Find **one real CVE** in a DNS server or a TLS library (search cve.org or nvd.nist.gov, e.g. for "BIND" or "OpenSSL") and give its ID. In 3–4 sentences: what broke, and what could an attacker do?

PART C — HANDS-ON EVIDENCE

Screenshots or command output from your own machine. You can reuse your Day 3 and Day 4 screenshots, but they must appear in the PDF:

- your IP, MAC and default gateway;
- a `ping` and a `tracert` (Windows: `tracert`) to any public site;
- one web request viewed in your browser's DevTools → Network tab (show the request headers).

PART D — REFLECTION

3–4 sentences: what clicked this week, and what's still fuzzy.

PART E — THINK LIKE AN ATTACKER

Take the login scenario from Part A and pick **three different places** where it could be attacked. For each, fill in one row:

<u>Attack point</u>	<u>Possible attack</u>	<u>Security control</u>
<u>(example) DNS</u>	<u>DNS spoofing sends you to an impostor</u>	<u>DNSSEC / secure DNS</u>

The three attack points must be from **different stages** of your Part A story. Don't copy the example — DNS spoofing is already used, so pick other attack points.

HOW IT'S MARKED (10 MARKS)

Area	What earns the marks	Marks
Technical accuracy	Part A is technically correct and in your own words	2
End-to-end explanation	All eleven stages present, in order, one coherent story	2
Security analysis	Security considerations in Part A + a sound threat model in Part E	2
R&D / research	Part B shows real research beyond class, correctly explained	2
Hands-on evidence	Part C: all three pieces present and legible	1
Reflection	Part D: genuine, not filler	1
Total		10

[Late or not one PDF? Talk to your instructor before the deadline.](#)

Submission checklist

[In your repo](#) (each folder holds the day's Do, Investigate, Secure and Evidence)

- ☐ [Repo created, private, named `cybersec-course`](#)
- ☐ [README.md has: why-I'm-here paragraph, the authorization sentence, "today I learned" list](#)
- ☐ [_day01/ — asset table](#)
- ☐ [_day02/ — pipeline diagram, program-vs-process paragraph, browser-process screenshot, security answer](#)
- ☐ [_day03/ — home-network drawing, four questions, `whatsmyip` answer, `ip a / traceroute` screenshots](#)
- ☐ [_day04/ — 7-step story, request-dissection table, blurred screenshot, padlock answer](#)
- ☐ [_day05/ — 5-primitives table, certificate + hash screenshots, password answer](#)

- ☐ Every day has a “Real job” answer and a “today I learned” entry

Handed in

- ☐ Authorization Pledge signed (paper, or scanned into the repo)
- ☐ Weekend assignment, Parts A–E in **one PDF**, submitted before Monday